

CSE 331

AUTOMATA AND COMPUTABILITY

PDA's

↘ pushdown automata (PDA)

Summer 2026

Today's learning goals

Sipser Ch 1.4, 2.1

- Define push down automata
- Trace the computation of a push down automaton
- Design a push down automaton recognizing a given language
- Compare class of regular languages and class of CFLs

RE
↓

DFA/NFA

CFLs
↓

PDA

Birds' eye view

All languages over Σ

Context-free languages over Σ

→ PDA

Regular languages over Σ

→ DFA/NFA

States = memory

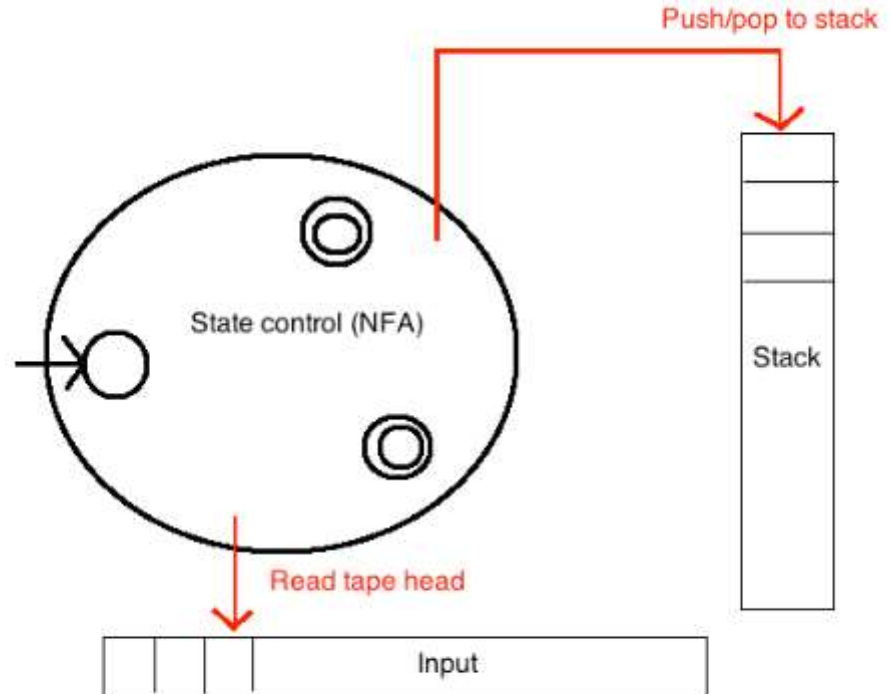
Finite languages over Σ

An alternative ...

Sipser p. 109

- NFA + stack = PDA

$a^n b$



Pushdown automata

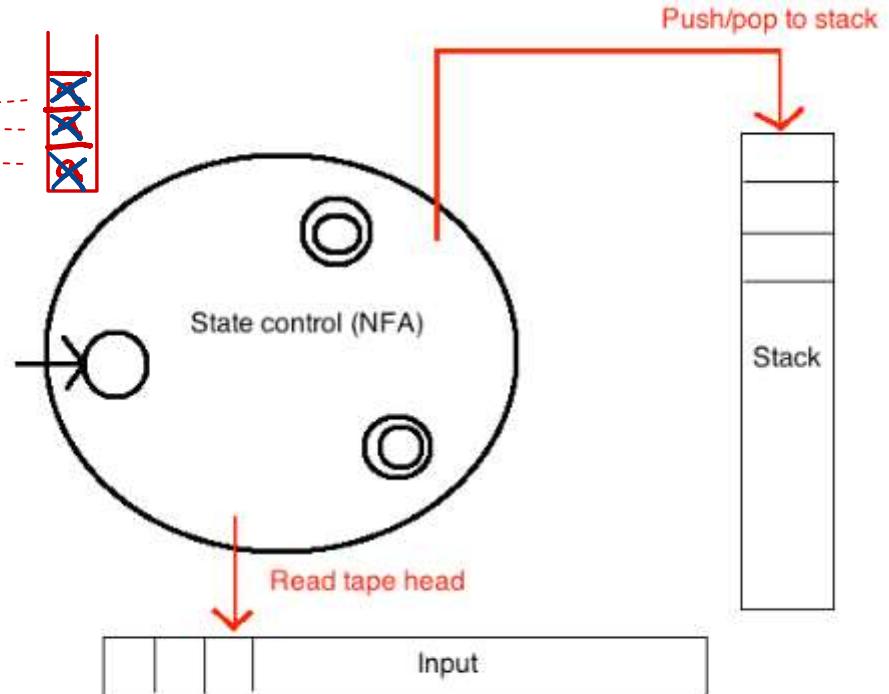
Sipser p. 109

- NFA + stack

$a^n b^n$
āāā bbb

At each step

1. **Transition** to new state based on current state, letter read, **and** top letter of stack.
2. (Possibly) **push or pop** a letter to (or from) top of stack



Pushdown automata

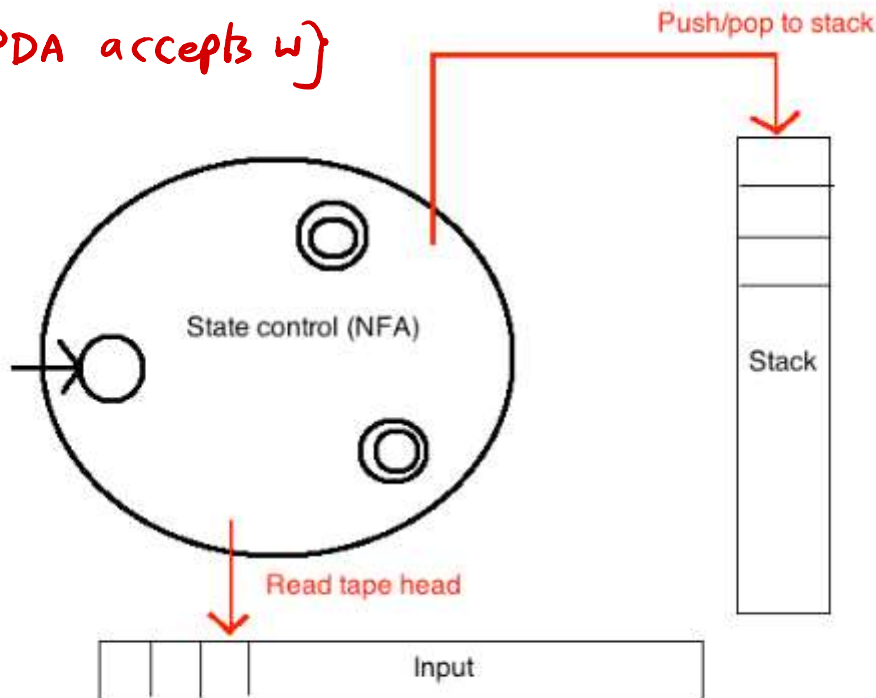
Sipser p. 109

- NFA + stack

$$L = \{w \mid \text{PDA accepts } w\}$$

Accept a string if

there is **some** sequence of states and **some** sequence of stack contents which processes the entire input string and ends in an accepting state.



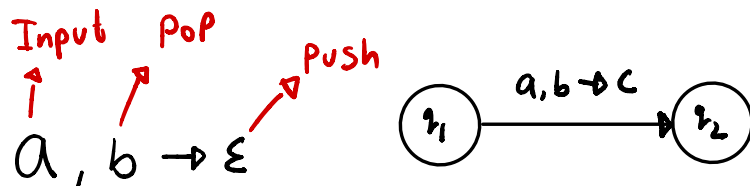
State diagram for PDA

Sipser p. 109

If hand-drawn or in Sipser

State transition labelled $a, b \rightarrow c$ means

"when machine reads an a from the input and the top symbol of the stack is a b , it may replace the b with a c ."



a on input

b on top of stack

Pop b , but don't push anything.

$a, \epsilon \rightarrow c$

a on input

Ignore top of stack

Pushing c .

State diagram for PDA

Sipser p. 109

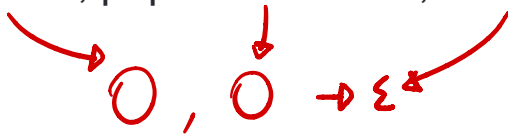
If hand-drawn or in Sipser

State transition labelled $a, b \rightarrow c$ means

"when machine reads an a from the input and the top symbol of the stack is a b , it may replace the b with a c ."

What edge label would indicate "Read a 0, pop 0 from stack, don't push anything to the stack"?

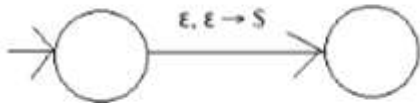
- A. $0, 0 \rightarrow \epsilon$
- B. $\epsilon, 0 \rightarrow 0$
- C. $0, \epsilon \rightarrow 0$
- D. $\epsilon \rightarrow \epsilon, 0$
- E. I don't know.



Useful trick

What would $\epsilon, \epsilon \rightarrow \$$ mean?

- A. Without reading any input or popping any symbol from stack, push \$
- B. If the input string and stack are both empty strings, push \$
- C. At the end of reading the input string, push \$ to top of stack
- D. I don't know.



Why is this useful?

Commonly used from initial state (at start of computation) to record top of stack with a special symbol)

"If the stack becomes empty and ..."

The L generated by $CFG \Leftrightarrow L$ recognized by PDA

Non-deterministic

Formal definition of PDA

Sipser Def 2.13 p. 111

A PDA is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$ where Q, Σ, Γ, F are all finite sets and

PDA = NFA + Stack

1. Q is the set of states $\rightarrow$ has multiple Computational Path.

2. Σ is the input alphabet

3. Γ is the stack alphabet $\rightarrow$ gamma Typically $\Sigma \subseteq \Gamma$

4. $\delta : Q \times \Sigma_{\epsilon} \times \Gamma_{\epsilon} \rightarrow \mathcal{P}(Q \times \Gamma_{\epsilon})$ is the transition function

5. $q_0 \in Q$ is the start state

$\Gamma_{\epsilon} = (\Gamma \cup \epsilon)$

6. $F \subseteq Q$ is the set of accept states.

Uses a power set because a PDA can have multiple possible transitions from current state.

$\delta(q_1, a, b) = \{(q_2, c), (q_3, d) \dots\}$

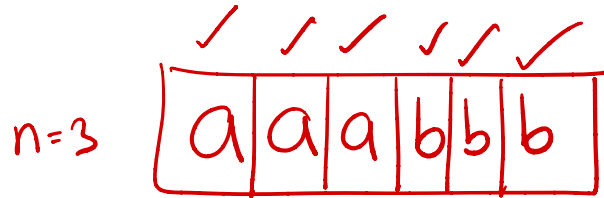
$\downarrow$ Currently in q_1 $\downarrow$ Read a from input $\downarrow$ Pop b from stack $\downarrow$ Push c to stack $\downarrow$ Move to q_2

Design a PDA

Transition: Read, pop $\rightarrow$ Push

Build a PDA that accepts the language

$$\{a^n b^n \mid n \in \mathbb{N}\}$$

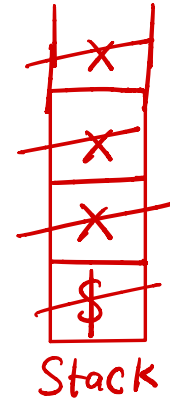


Input tape

$a, \epsilon \rightarrow X$

$b, X \rightarrow \epsilon$

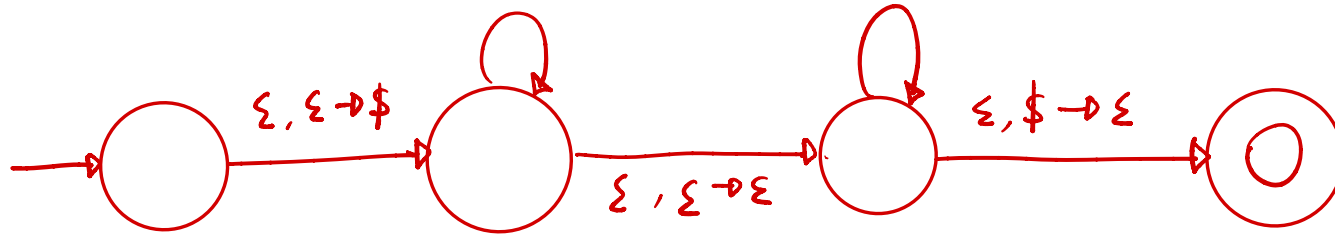
$$\Gamma = \{X, \$\}$$



$n=0$

(ϵ)

$\{0, 1, 2, 3, \dots\}$



Designing a PDA

$$L = \{ 0^i 1^{i+1} \mid i \geq 0 \}$$

Informal description of PDA:

Read symbols from the input. As each 0 is read, push it onto the stack. As soon as 1s are seen, pop a 0 off the stack for each 1 read. If the stack becomes empty and there is exactly one 1 left to read, read that 1 and accept the input. If the stack becomes empty and there are either zero or more than one 1s left to read, or if the 1s are finished while the stack still contains 0s, or if any 0s appear in the input following 1s, reject the input.

Designing/Tracing a PDA

If \$ popped from stack, the stack is empty.

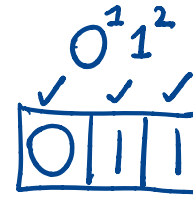
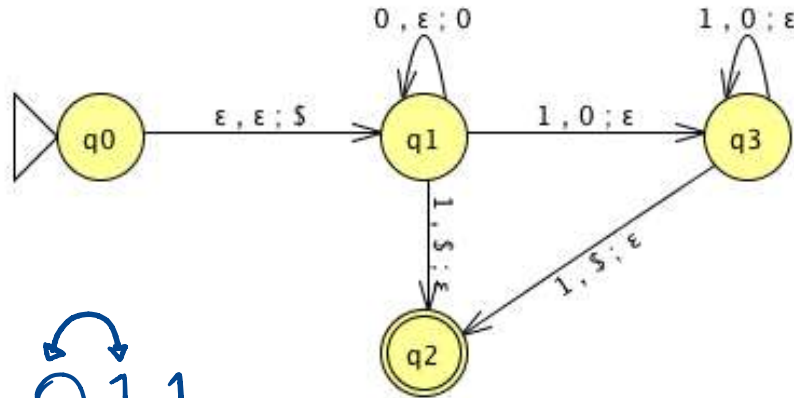
Transition: Read, pop $\rightarrow$ Push

$$L = \{ 0^i 1^{i+1} \mid i \geq 0 \}$$

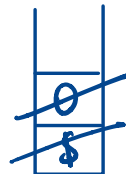
$i=1$

Sample computations?

- nondeterminism
- stack contents



CFG: $S \rightarrow 0S1 \mid 1$



Input	q	Stack
	q_0	—
	q_1	\$
0	q_1	0 \$
1	q_3	\$
1	q_2	

Designing a PDA

$$L = \{ 0^i 1^{i+1} \mid i \geq 0 \}$$

~~0001111~~

Which of these CFGs generate L?

Assume V = set of variables mentioned in rules; $\Sigma = \{0, 1\}$, S start

A. $S \rightarrow \varepsilon \mid 0S1$

~~B. $S \rightarrow \varepsilon \mid 0S11$~~

☒ C. $S \rightarrow T1 \mid 0S1, T \rightarrow \varepsilon$

~~D. $S \rightarrow \varepsilon \mid 0T1, T \rightarrow \varepsilon \mid 0T1$~~

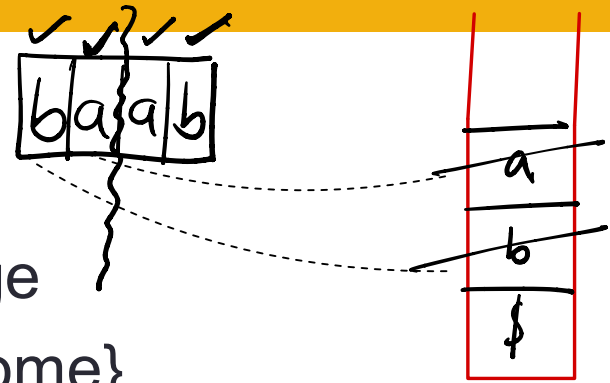
E. None of those

$S \rightarrow \underline{T1} \mid 0\underline{S1}$

$T \rightarrow \varepsilon$

Design a PDA

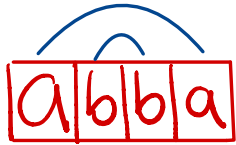
$abba$
 $\longleftrightarrow$



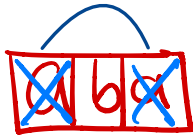
Build a PDA that accepts the language

$$\Sigma = \{a, b\} \quad \{ w \mid w \text{ is a palindrome} \}$$

Read from front = Read from back



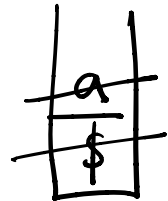
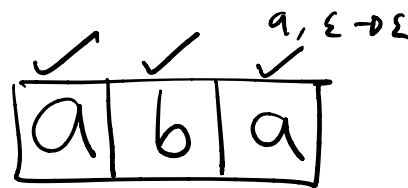
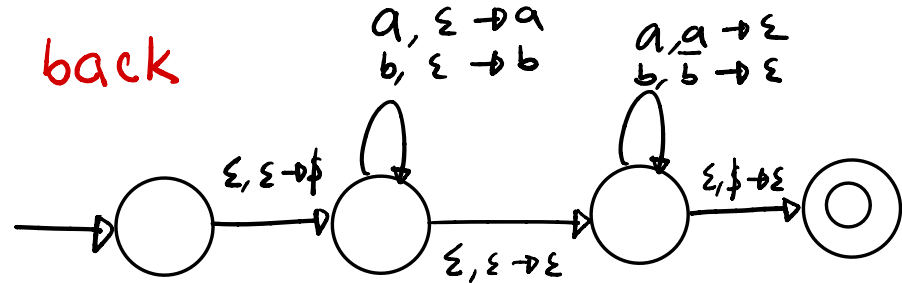
even length
Palindrome



odd length
Palindrome

$$S \rightarrow aSa \mid bSb \mid a \mid b \mid \epsilon$$

Base Case



Design a PDA

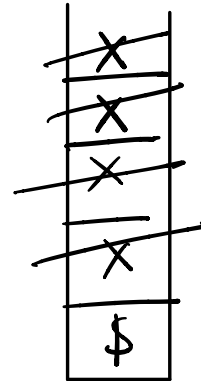
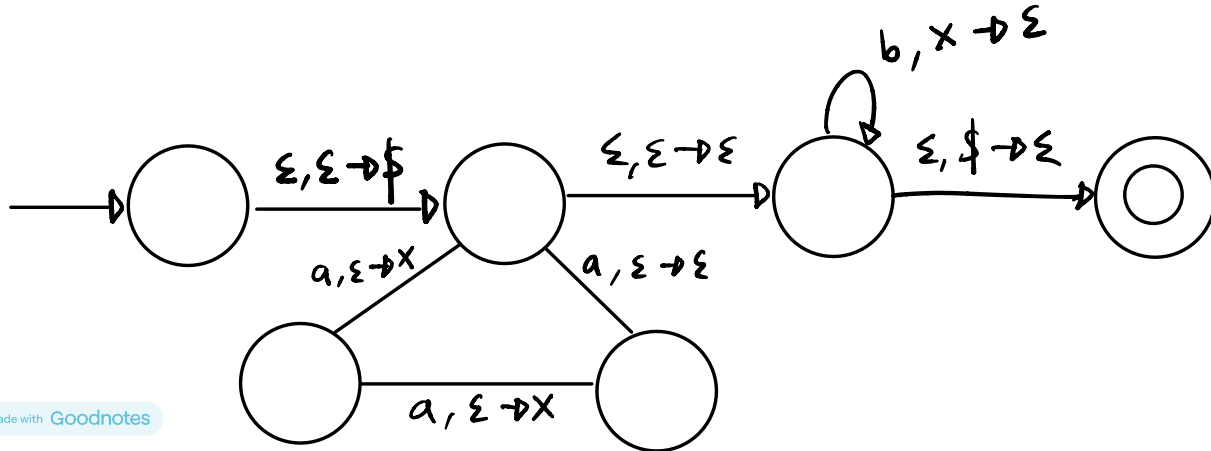
Read 3 a's : Push two X to stack
 Read 2 b's : Pop two X from stack

Build a PDA that accepts the language

Transition: Read, pop $\rightarrow$ Push

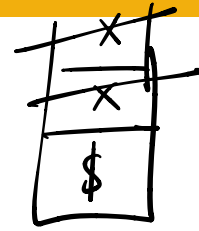
$$\{ a^{3n} b^{2n} \mid n \in \mathbb{N} \}$$

$$L = \left\{ \begin{array}{l} \epsilon \\ n=0 \end{array} , \begin{array}{|c|c|c|c|} \hline a & a & a & b & b \\ \hline \end{array} \begin{array}{l} n=1 \\ \text{---} \end{array} , \begin{array}{c} \text{---} \\ n=2 \end{array} \right\}$$



Design a PDA

aa b b

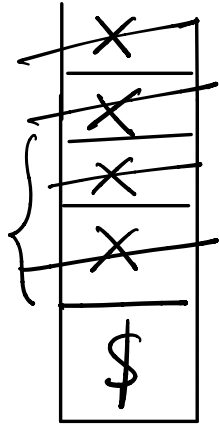


Build a PDA that accepts the language

$$\{a^n b^m \mid n > m \text{ and } m, n \in \mathbb{N}\}$$

$$a^n b^m$$

extra
a's



aa
a > b
#a = #b

